

Report - Kaggle Regression

Khiem Nguyen

07/20/26

Contents

Introduction	2
Exploratory Data Analysis	2
1) <i>Monthly Revenue Over Time</i>	2
2) <i>Distribution of $\log_{10}(\text{Monthly Revenue})$</i>	2
3a) <i>Top 15 States by Total Revenue</i>	3
3b) <i>Top 15 States by Revenue per Customer</i>	3
4a) <i>Revenue per Customer by Age</i>	4
4b) <i>Revenue per Customer by Income Bracket</i>	4
4c) <i>Revenue per Customer by Age $\times$ Income</i>	5
5) <i>Revenue per Customer by Gender</i>	6
6) <i>Revenue per Customer by Order Frequency</i>	6
7a) <i>Top 20 Categories by Revenue</i>	7
7b) <i>Top Categories by Age Band</i>	7
7c) <i>Category Share by Year</i>	8
8) <i>Revenue per Customer by Household Size</i>	8
9) <i>Distribution of $\log_total$ (from training set)</i>	8
10) <i>Spearman Correlation Heatmap (from training set)</i>	9
11) <i>$\log_total$ by State (from training data)</i>	10
12) <i>Selected feature vs $\log_total$</i>	10
Preprocessing / Recipes	11
Candidate Models	11
Model Evaluation and Tuning	13
Discussion of Final Model	14
Appendix: Final Annotated Script	14

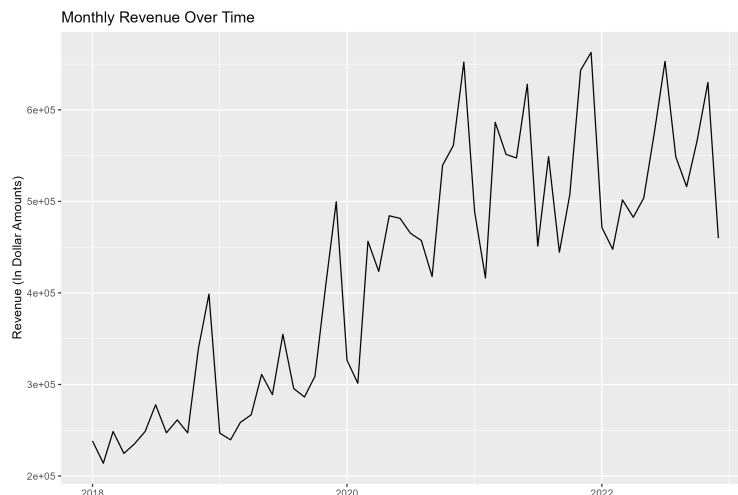
Introduction

In this study, we are trying to predict the log of the total revenue from Amazon orders in each state, given a month and year. Based on background knowledge, we believe several given factors are likely to be associated with the response variable, 'log_total'. The month of each observation may be related to its 'log_total', since seasons, including summer and the holidays, would expectably cause an increase in orders. 'count' should also be associated with 'log_total', since more customers generally translate to higher total revenues. The customer income variable would also be a factor in determining 'log_total', since households with higher disposable incomes can spend more on their Amazon purchases. Additional predictors may include 'age', as well as 'household size' and 'gender'.

Exploratory Data Analysis

1) Monthly Revenue Over Time

```
knitr::include_graphics("figs/01_monthly_revenue_over_time.png")
```

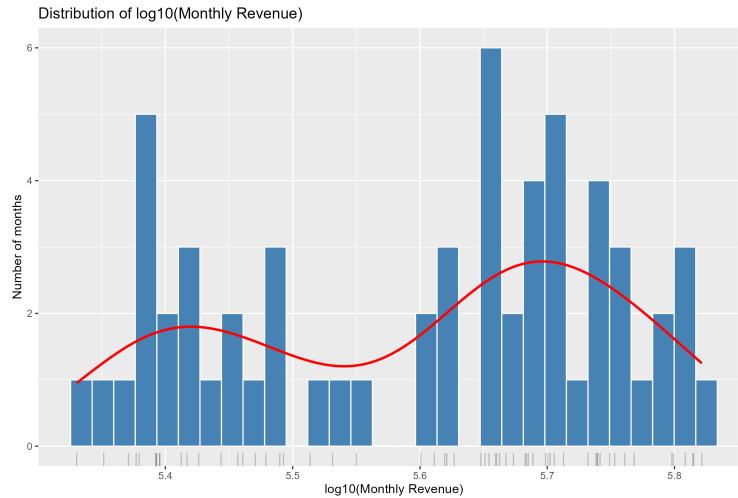


Monthly revenue gradually rose from 2018 to 2022, with clear seasonal spikes late each year and temporary dips following the holiday seasons. The upward trend possibly reflects growth in customers and order volumes, while the recurring peaks show the trends of seasonality. **Prediction model should show an upward trend in revenue over time, with peaks during the holidays followed by troughs.**

*This plot does not account for higher-priced items or general inflation.

2) Distribution of $\log_{10}(\text{Monthly Revenue})$

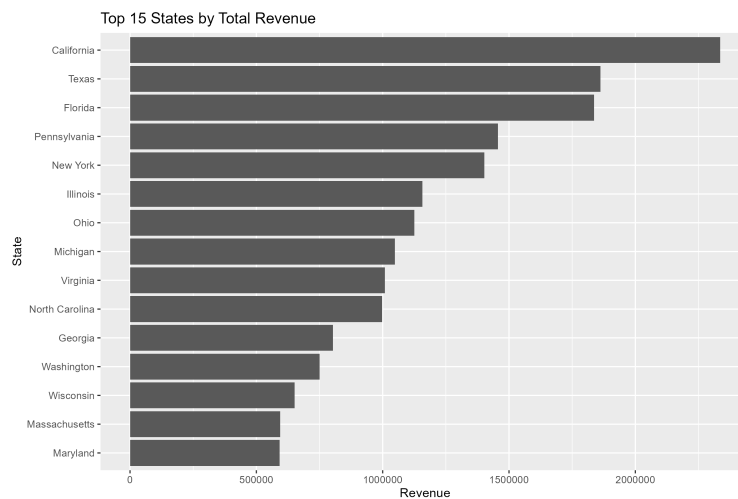
```
knitr::include_graphics("figs/02_distribution_of_log10.png")
```



Monthly revenue (log10) clusters between 5.6 and 5.8, with a secondary mass around 5.4 and 5.5. The log transformation reduces skewness and makes visible the seasonal highs versus off-peak months. **Modeling should allow for seasonal effects instead of assuming a single average month.**

3a) Top 15 States by Total Revenue

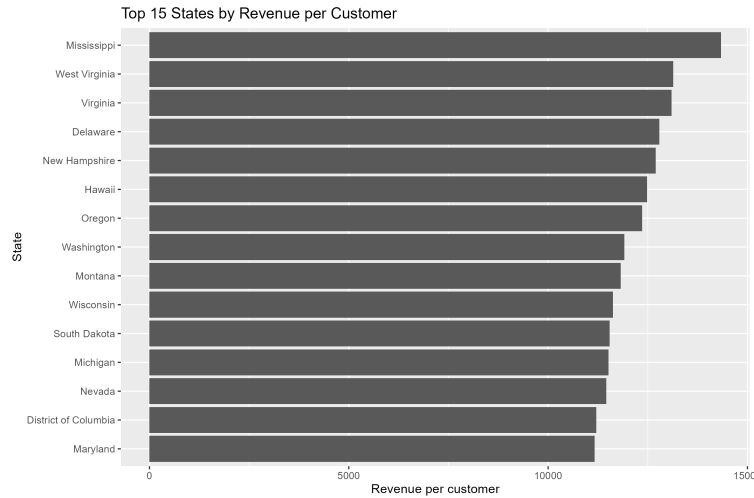
```
knitr::include_graphics("figs/03_top_15_states_by_total_revenue.png")
```



This graph simply shows the geographic concentration of spending and confirms the state-level imbalance, but doesn't suggest new predictive power beyond that factor.

3b) Top 15 States by Revenue per Customer

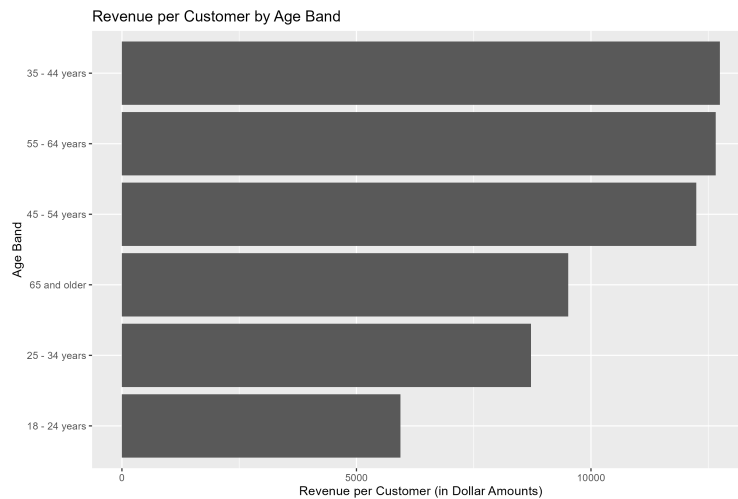
```
knitr::include_graphics("figs/04_top_15_states_by_revenue_per_customer.png")
```



This one complements the graph above. Smaller states like Mississippi, West Virginia, and Delaware show the highest revenue generated for each customer. **Random Forests/XGB can split on state to learn “all else equal, Mississippi is more active than California.”**

4a) Revenue per Customer by Age

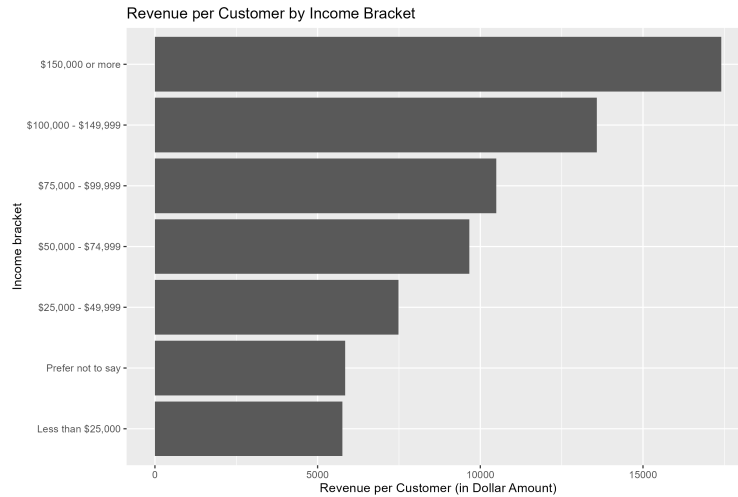
```
knitr::include_graphics("figs/05_revenue_per_customer_by_age_band.png")
```



Customers aged 35-44 spend the most on average, followed closely by 55-64 and 45-54. This suggests that spending peaks in mid-life stages. **Including these factors in our model will let it learn how customer age bands affect revenue. Tree models do especially better than linear models in capturing this curved trend.**

4b) Revenue per Customer by Income Bracket

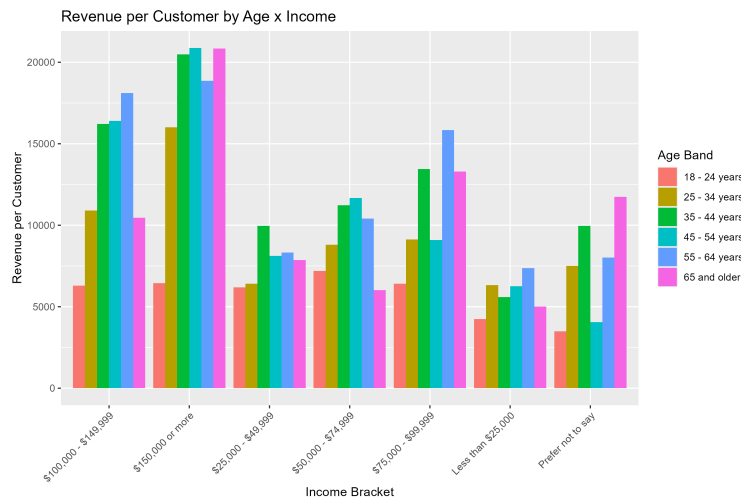
```
knitr::include_graphics("figs/06_revenue_per_customer_by_income_bracket.png")
```



Higher-income groups, \$100k-149k and \$150k+, spend substantially more per customer than lower-income and “prefer not to say” groups. **The positive linear relationship between income groups and revenue per customer should be reflected in predictions. We should also keep in mind that income effects are strong but may interact with other demographics (e.g., high-income young vs. older households).**

4c) Revenue per Customer by Age x Income

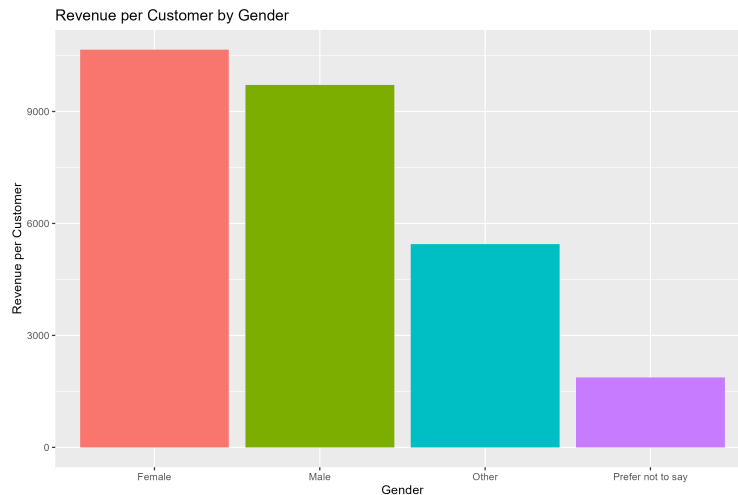
```
knitr::include_graphics("figs/07_revenue_per_customer_by_age_x_income.png")
```



The revenue per customer rises with income across all ages. However, the differences in revenue per customer between age groups grow wider at higher income brackets; the widening gaps confirm that age and income interact. **Tree-based methods like Random Forests can capture the interaction because they learn conditional splits automatically.**

5) Revenue per Customer by Gender

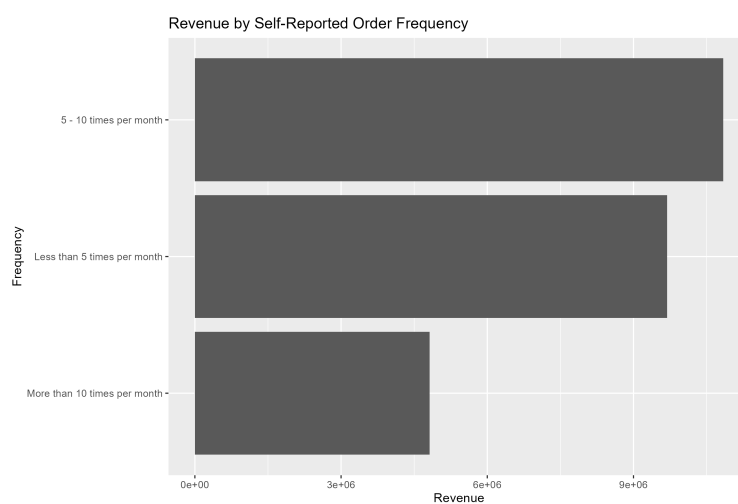
```
knitr::include_graphics("figs/08_revenue_per_customer_by_gender.png")
```



Female customers spend the most per person, follow by males. **A Random Forest or XGB model can naturally learn that a higher share of females at an observation predicts higher log revenue. Splitting the training set on count_female / count will capture the per-customer spending pattern effectively.**

6) Revenue per Customer by Order Frequency

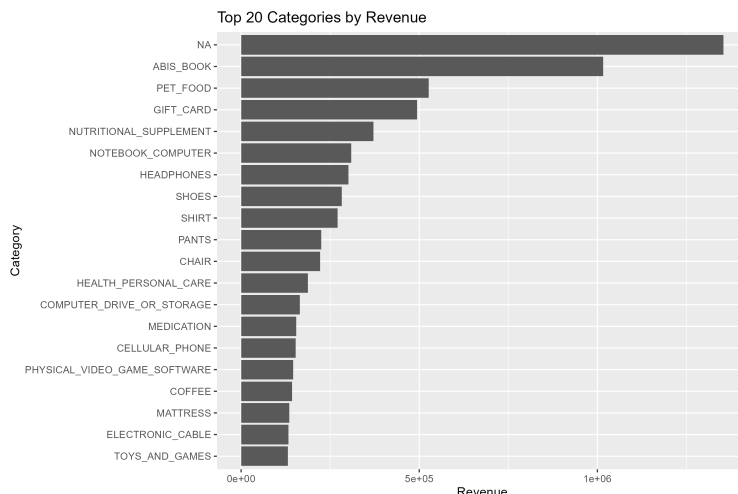
```
knitr::include_graphics("figs/09_revenue_by_self_reported_order_frequency.png")
```



This one is self-explanatory. It simply confirms that order frequency reflects the intensity of spend. **Tree models will learn that higher shares of high-frequency users at an observation boost log_total.**

7a) Top 20 Categories by Revenue

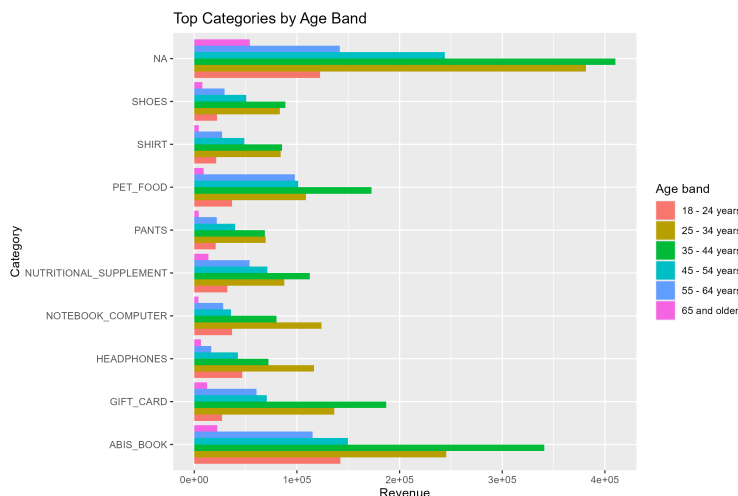
```
knitr::include_graphics("figs/10_top_20_categories_by_revenue.png")
```



Books, pet foods, and gift cards are the categories yielding the highest revenue by a big margin, compared to the other categories. **Product categories don't appear directly in the training set, but it's still gives context for the dataset.**

7b) Top Categories by Age Band

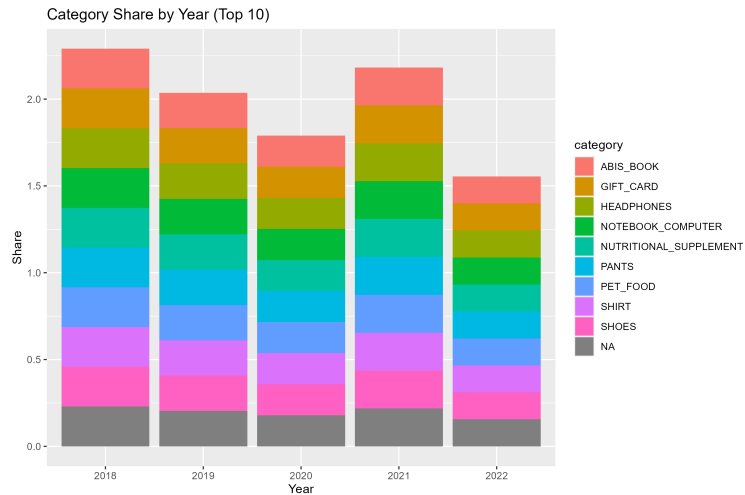
```
knitr::include_graphics("figs/11_top_20_categories_by_age_band.png")
```



Certain age groups skew towards certain categories. For example, books and gift cards show strong contributions from 25-44 year olds, while nutritional supplements are popular among older generations. **Again, this information adds context to the data and its trends, but it does not add predictive value.**

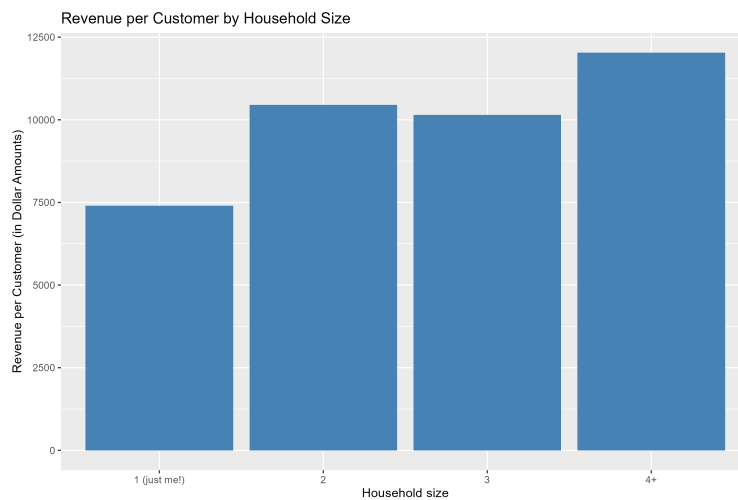
7c) Category Share by Year

```
knitr::include_graphics("figs/12_category_share_by_year.png")
```



8) Revenue per Customer by Household Size

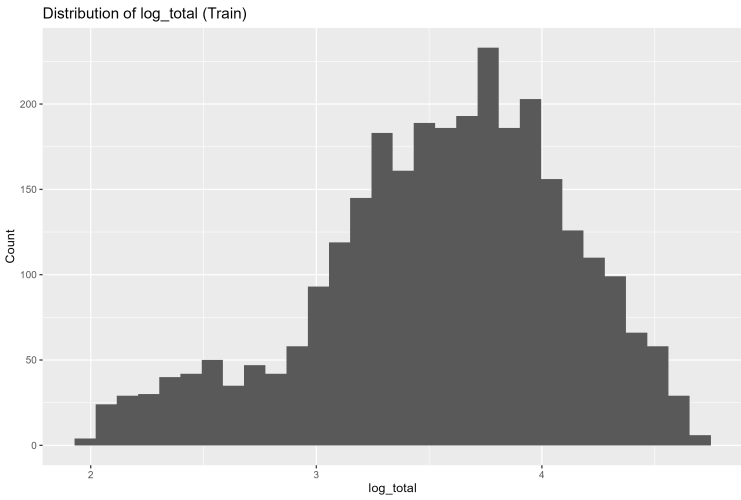
```
knitr::include_graphics("figs/13_revenue_per_customer_by_household_size.png")
```



Customers in larger households spend the most per person, while single-person households spend the least. This suggests a predictable, positive linear relationship between household size and per-customer revenue. **A Random Forest or XGB model can split on household composition, and it may find that observations with a higher share or large households will yield higher log revenues.**

9) Distribution of log_total (from training set)

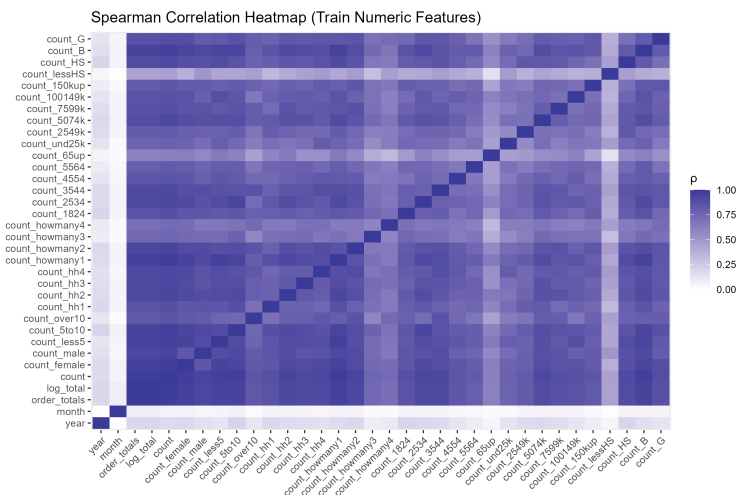

```
knitr::include_graphics("figs/14_distribution_of_log_total.png")
```



The target variable, monthly log revenue at the state-month level, follows roughly a left-skewing, bell-shaped distribution. **Since this is the target that the models are trained to predict later, its distribution is important.** Tree-based models thrive on well-behaved data like the one shown. For linear regression, the transformation is critical.

10) *Spearman Correlation Heatmap (from training set)*

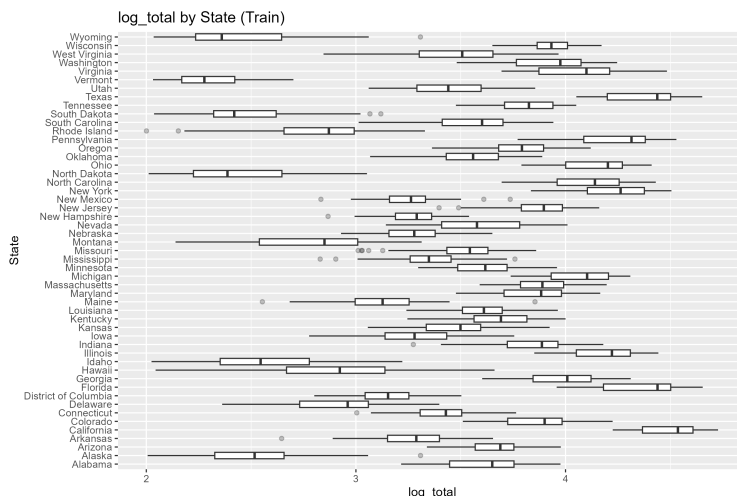
```
knitr::include_graphics("figs/15_spearman_correlation_heatmap.png")
```



Many demographic subcounts are strongly correlated with one another; for example, “count-female” is highly correlated with count, and age groups move in opposite directions. **These correlations highlight the weaknesses of linear regression. Multicollinearity inflates coefficient variance, unless a good amount of transformations are performed. Tree-based models handle redundancy more effectively.**

11) *log_total by State (from training data)*

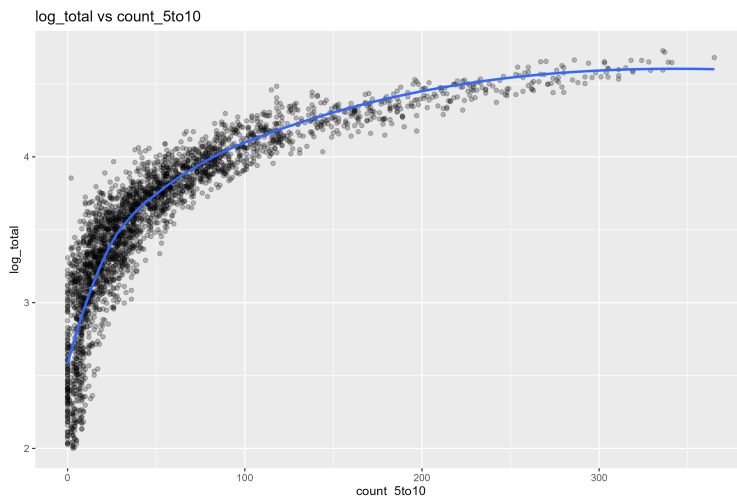
```
knitr::include_graphics("figs/16_log_total_by_state.png")
```



Revenue levels vary a lot across states. Larger states like California and New York show higher medians and larger spreads, compared to the smaller or less populous states. **Tree-based models can split on specific states and capture state-specific effects (when they outperform or underperform), after adjusting for demographics.**

12) *Selected feature vs log_total*

```
knitr::include_graphics("figs/17_selected_feature_vs_log_total.png")
```



There is a clear nonlinear positive relationship. As the number of medium-frequency customers increases, total log revenue rises with diminishing returns. **This confirms that count_5to10 is a strong predictor of revenue. Tree-based models can capture thresholds, which makes count_5to10 especially valuable.**

Preprocessing / Recipes

Our first stage of preprocessing variables is removing the `order_totals` variable because `order_totals` perfectly predicts `log_total` since it was calculated from it. For our linear model, first two random forest ranger engine models, and our XGB model, our only preprocessing step is to create a recipe that predicts `log_total` off of all other variables. For our random forest using linear regression model, our recipe dropped zero variance predictors, converted raw count variables into proportions based on count, stabilized the scaled proportions with a log transform of count, dropped raw counts after making ratios, and collapsed rare categories under a 1% threshold.

Candidate Models

Our first model is a simple linear model, which predicts `log_total` based on all other variables in the data.

Our second candidate model is a linear regression that includes interaction terms identified in the EDA. We suspected that age interacts with income and that household size also interacts with income.

Our third candidate model is a random forest model using the ranger engine. We set its arguments to `trees = 3000`, `mtry = 2`, and `min_n = 5`.

Our fourth candidate model has the same mechanism as the second, but we changed the hyperparameter from 2 to 3 instead. This was to test if more randomized splits and smaller sample fractions would improve generalization.

Our fifth candidate model is a random forest using a linear regression engine. This model attempts to drop unimportant noise from the data, then scales the remaining predictors. We added the following specifications to our engine: `importance = "none"`, `splitrule = "extratrees"`, `replace = FALSE`, and `sample.fraction = 0.7`.

Our sixth candidate model is a boosted trees model whose hyperparameters (`tree_depth`, `learn_rate`, `mtry`, `min_n`, and `loss_reduction`) are found during tuning. We set the trees during training to 500 to be lighter for cross-validation; while for our test data, we set it to 3000 for a heavier fit.

```
include_graphics("model_table.png")
```

Candidate Models				
Model.Identifier	Type.of.Model	Engine	Recipe	Hyperparameters
Normal Linear Regression	Linear Regression	lm	recipe(log_total ~ .)	N/A
Normal Linear Regression w/ Interaction Terms	Linear Regression	lm	recipe(log_total ~ .) %>% step_dummy(all_nominal_predictors()) %>% step_interact(terms = ~ age x income + household_size x income) %>% step_zv(all_predictors())	N/A
Random Forest 1	Random Forest	ranger	recipe(log_total ~ .)	trees = 3000, mtry = 3, min_n = 5
Random Forest 2	Random Forest	ranger	recipe(log_total ~ .)	trees = 3000, mtry = 2, min_n = 5
Random Forest (extratrees, sub-sampling)	Random Forest	ranger	recipe(log_total ~ ., data = train) %>% step_zv(all_predictors()) %>% step_ratio(starts_with("count_"), denom = denom_vars(count)) %>% step_mutate(ln_count = log1p(count)) %>% step_rm(starts_with("count_")) %>% step_other(all_nominal_predictors(), threshold = 0.01)	mtry = tune(), min_n = tune(), trees = 500
XGBoost Gradient Boosted Trees	Boosted Trees (regression)	xgboost	recipe(log_total ~ .)	trees = 500, tree_depth = tune(), learn_rate = tune(), mtry = tune(), min_n = tune(), loss_reduction = tune()

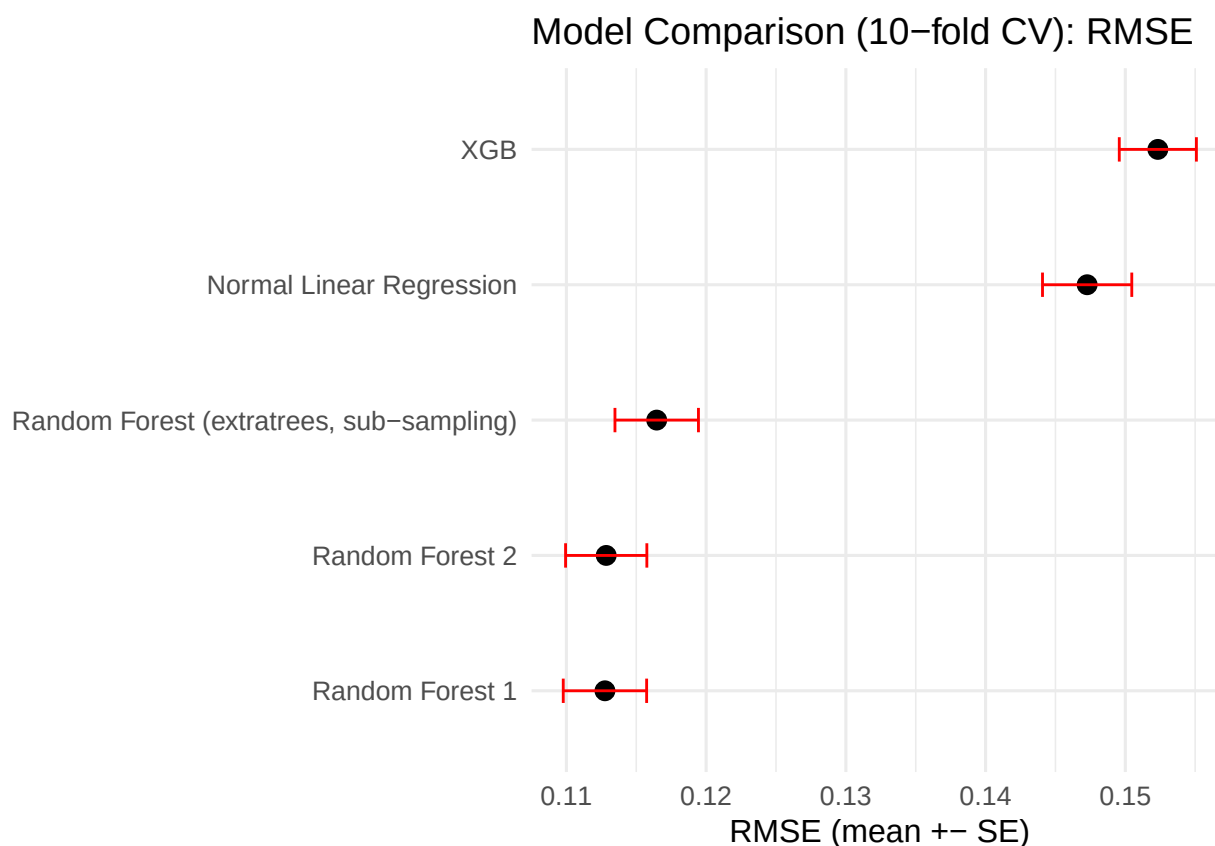
Model Evaluation

Model.Identifier	Metric.Score	SE.of.Metric
Normal Linear Regression	0.1472681	0.003194044
Normal Linear Regression w/ Interaction Terms	0.1250000	0.002110000
Random Forest 1	0.1127591	0.002984036
Random Forest 2	0.1128492	0.002908474
Random Forest (extratrees, sub-sampling)	0.1164627	0.002987771
XGBoost Gradient Boosted Trees	0.1523278	0.002759284

Model Evaluation and Tuning

All models were evaluated using 10-fold cross validation stratified on `log_total`, with a seed of 123. RMSE was used as the model evaluation metric. Hyperparameters were tuned for Model 4 (Random Forest with `extratrees`), and Model 5 (XGBoost) via racing ANOVA and grid search.

`model_comparison`



`model_eval`

Based on the results of the RMSE metric scores, we can make many observations and comparisons of the five models. First, Model 2 and Model 3 achieved the lowest RMSE and outperformed all other models.

Model 4 showed no improvement relative to the simpler Random Forest models. Model 1 (Linear Regression) underfit the data, with a higher RMSE of 0.147. Finally, Model 5 (XGBoost) underperformed despite the hyperparameter tuning.

Discussion of Final Model

The runner-up was Model 3, Random Forest 2 with `mtry` set to 3. The results suggest that Random Forest is well-suited to capture the non-linearities and complex interactions present in the data, while still maintaining a strong generalization. The strengths of Model 2 include consistently producing the lowest RMSE among the candidate models and performing well with high-dimensional data and noisy predictors.

Generally, we can see that a strength of the Random Forest models was that they were generally stable across different `mtry` values. A drawback of Model 2 is the limited interpretability the model provides. Random Forest models do not give interpretable coefficients, making it difficult to estimate the effects of the individual predictors. Furthermore, the final model was computationally expensive and slow when compared to a simpler linear regression model. With this model, a key component to mitigate the risk of overfitting was carefully controlling the parameters and tuning.

While Random Forest already handles nonlinearities relatively well, engineering additional features, with information about the context of the data, could help the trees better learn. Creating ratios, log transformations, or interaction terms could potentially reduce RMSE even further.

Appendix: Final Annotated Script

```
library(rsample)

set.seed(123) # set a random seed

# recipe, normalizing all numeric predictors
rf_rec <- recipe(log_total ~., data = train) %>%
  step_normalize(all_numeric_predictors())

# defining random forest model
rf_model <- rand_forest(
  trees = 3000, # should be sufficient
  mtry = 2,
  min_n = 5
) %>%
  set_engine("ranger") %>%
  set_mode("regression")

# recipe + model
rf_wf <- workflow() %>%
  add_model(rf_model) %>%
  add_recipe(rf_rec)

# stratify
train_folds <- vfold_cv(train, v = 10, strata = log_total)

# cross validation for evaluation
rf_fit <- fit_resamples(
  rf_wf,
```

```

    resamples = train_folds,
    metrics = metric_set(rmse)
)

# collect metrics from CV results
metrics <- collect_metrics(rf_fit)

# filter for rmse, then also calculate mse manually (just squaring mean rmse)
metrics %>%
  filter(.metric == "rmse") %>%
  mutate(mse = mean^2)

# fit the final model on the full training data
rf_fit_final <- rf_wf %>%
  fit(data = train)

# make predictions on test set
pred <- test %>%
  select(id) %>%
  bind_cols(
    predict(rf_fit_final, new_data = test)
  )

# format predictions for submission (id + log_total)
pred %>%
  rename(log_total = .pred) %>%
  select(id, log_total) %>%
  write_csv("rf_fit_final_pred_2.csv")

```